Prompt Engineering Deep Dive

The Art & Science of Effective Prompts

Day 2 - Session 1

KavinSchool

Day 2: What We'll Cover Today

Session	Topic	Key Outcome
Session 1	Prompt Engineering Fundamentals	Write clear, effective prompts
Session 2	Prompting Techniques	Zero-shot, few-shot, CoT, role prompting
Session 3	API Parameters & Output Control	Master temperature, top_p, streaming
Session 4	Iteration, Refinement & Lab	Build a refactor assistant

> **Theme:** Move from "it works" to "it works reliably."

Quick Day 1 Recap

Yesterday you learned:

1. **LLMs predict the next token.** They don't truly "understand"
2. **The API is stateless.** Send full conversation each turn
3. **Tokens = currency.** Billing, context limits, everything is tokens
4. **System messages shape behavior.** Invest time crafting them
5. **Model selection matters.** Balance capability, cost, and privacy

> Today we master the **craft** of writing prompts that produce reliable, high-quality output.

What Makes a Prompt "Good"?

From Vague Requests to Precise Instructions

KavinSchool

Good Prompts vs Bad Prompts

Bad prompt:

```
Tell me about Python
```

Good prompt:

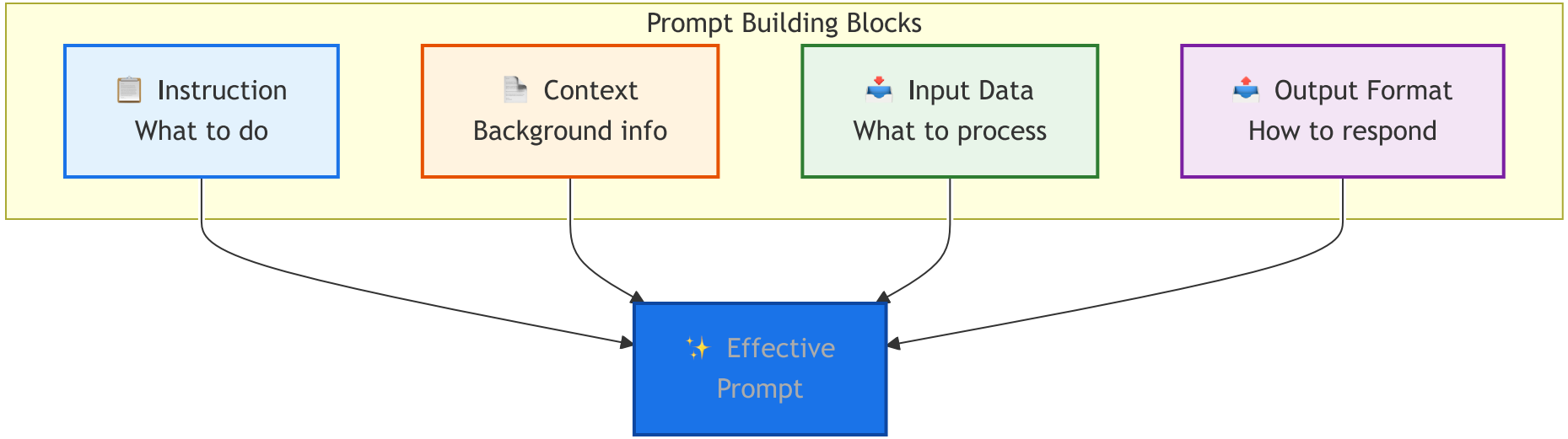
```
Explain Python's Global Interpreter Lock (GIL) to a developer
who knows Java threading. Include: what it is, why it exists,
how it affects multi-threaded Python programs, and two
workarounds. Keep it under 200 words.
```

> The difference? **Clarity, specificity, context, and constraints.**

The Four Qualities of Effective Prompts

Quality	What It Means	Example
Clarity	Unambiguous instruction	"Summarize" vs "Summarize in 3 bullet points"
Specificity	Narrow the scope	"Write code" vs "Write a Python function that validates email addresses using regex"
Context	Provide background	"You are reviewing code for a fintech startup with strict security requirements"
Constraints	Set boundaries	"Under 200 words", "Respond in JSON", "Use only standard library"

The 4 Building Blocks of a Prompt



Building Block 1: Instruction

The **core task** you want the model to perform.

```
❌ "Python decorators"
✅ "Explain Python decorators with a practical example"
✅ "List 5 common use cases for Python decorators in web development"
✅ "Review this code and identify any misuse of decorators"
```

Tips:

- Start with a verb: Explain, Summarize, Classify, Generate, Compare, Refactor
- Be specific about what "done" looks like
- One clear task per prompt (unless chaining is intentional)

Building Block 2: Context

Background information that shapes the response.

```
messages = [
    {"role": "system", "content":
        "You are a senior backend engineer at a healthcare startup. "
        "Code must be HIPAA-compliant. The team uses Python 3.12, "
        "FastAPI, and PostgreSQL. Follow PEP 8."},
    {"role": "user", "content":
        "Write an endpoint to store patient records."},
]
```

> Without context, the model gives generic answers. With context, it gives **relevant** answers.

Building Block 3: Input Data

The material the model should process.

```
messages = [
    {"role": "system", "content": "Summarize the key findings."},
    {"role": "user", "content": f"""
Summarize this research paper abstract:

---
{abstract_text}
---

Provide 3 key findings as bullet points.
"""},
]
```

Building Block 3: Input Data

Best practices:

- Use **delimiters** (`---`, `"""`, `###`) to separate input data from instructions
- Label sections clearly: "Input:", "Context:", "Document:"
- Remove noise before sending

Building Block 4: Output Format

Tell the model **how** to format the response.

```
messages = [
    {"role": "system", "content": """Respond with ONLY valid JSON:
{
    "sentiment": "POSITIVE | NEGATIVE | NEUTRAL",
    "confidence": 0.0 to 1.0,
    "key_phrases": ["phrase1", "phrase2"]
}"""},
    {"role": "user", "content": "The new feature is amazing but the app crashes too often."},
]
```

> Specifying output format is the single most impactful prompt improvement for production systems.

Putting It All Together

```
# All 4 building blocks in action
messages = [
    {"role": "system", "content":
        # CONTEXT
        "You are a senior Python code reviewer at a fintech company. "
        # CONSTRAINTS
        "Be concise. Focus on bugs, security, and performance."},
    {"role": "user", "content":
        # INSTRUCTION
        "Review the following function for issues.\n\n"
        # INPUT DATA
        f"```python\n{code_snippet}\n```\n\n"
        # OUTPUT FORMAT
        "Respond as a numbered list: issue, severity (HIGH/MEDIUM/LOW), fix."},
]
```

Defining Goals Before Prompting

Start With the End in Mind

KavinSchool

Before You Write a Prompt, Answer These

Question	Why It Matters
What is the **exact output** I need?	Prevents vague prompts
Who is the **audience**?	Shapes tone and detail level
What **format** should it be in?	JSON? Markdown? Plain text?
What **constraints** apply?	Length, style, allowed values
How will I **validate** the output?	Determines if the prompt is working
What happens if it **fails**?	Error handling strategy

> Write the expected output **first**, then work backwards to the prompt.

Example: Goal-First Prompting

Goal: Classify customer support tickets into categories.

Expected output:

```
{"category": "billing", "priority": "high", "requires_human": true}
```

Work backwards to the prompt:

```
system = """You are a customer support ticket classifier.
Classify each ticket into exactly one category:
billing, technical, account, feature_request, other.

Set priority: high, medium, low.
Set requires_human: true if the issue needs human intervention.

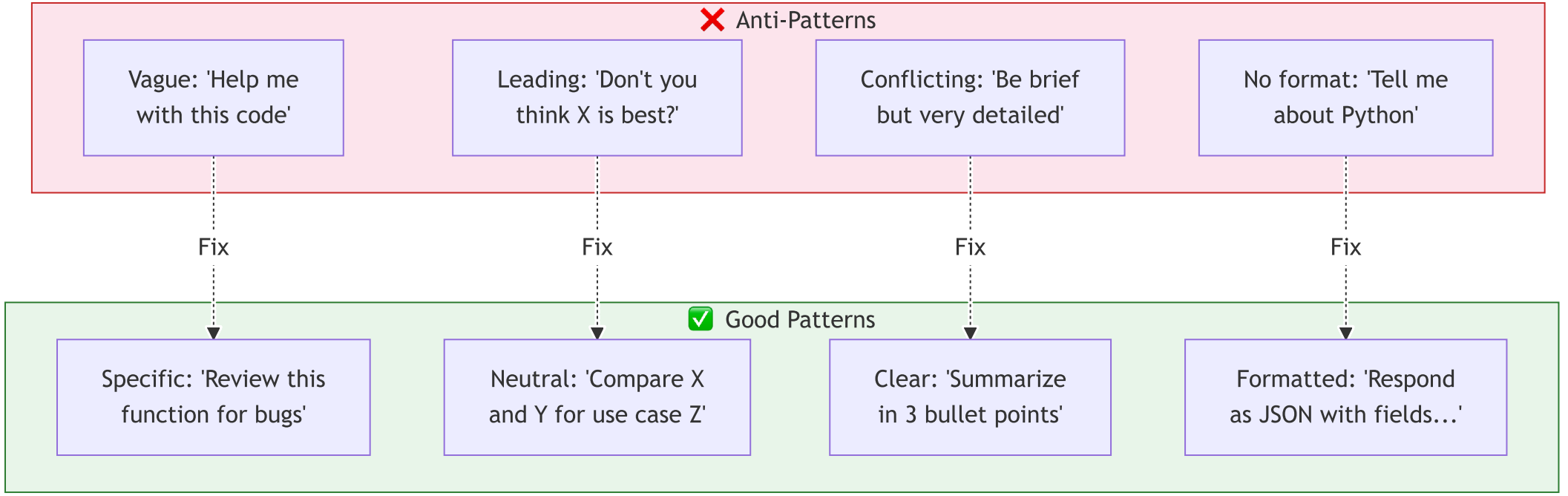
Respond with ONLY valid JSON. No other text."""
```

Common Anti-Patterns

What NOT to Do

KavinSchool

Prompt Anti-Patterns



Anti-Pattern 1: Vague Instructions

```
❌ "Help me with this code"
❌ "Make it better"
❌ "Fix the bug"
```

What does "better" mean? Faster? More readable? Shorter?

```
✅ "Refactor this function to reduce cyclomatic complexity.
    Split into smaller functions. Add type hints.
    Keep the public API unchanged."
```

Anti-Pattern 2: Leading Questions

❌ `"Don't you think Python is better than Java for this task?"`

The model will agree with you. It's trained to be helpful.

✅
```
"Compare Python and Java for building a REST API.
 Consider: development speed, performance, ecosystem,
 team expertise. Recommend one with justification."
```

Anti-Pattern 3: Conflicting Constraints

```
❌ "Be very brief but explain everything in detail."
❌ "Be creative but stick exactly to the template."
```

The model has to pick one. You might not like which one.

```
✅ "Provide a 3-sentence summary, then a detailed
    explanation in bullet points underneath."
```

Anti-Pattern 4: No Output Specification

❌ `"Analyze this data"`

You'll get a wall of text in unpredictable format.

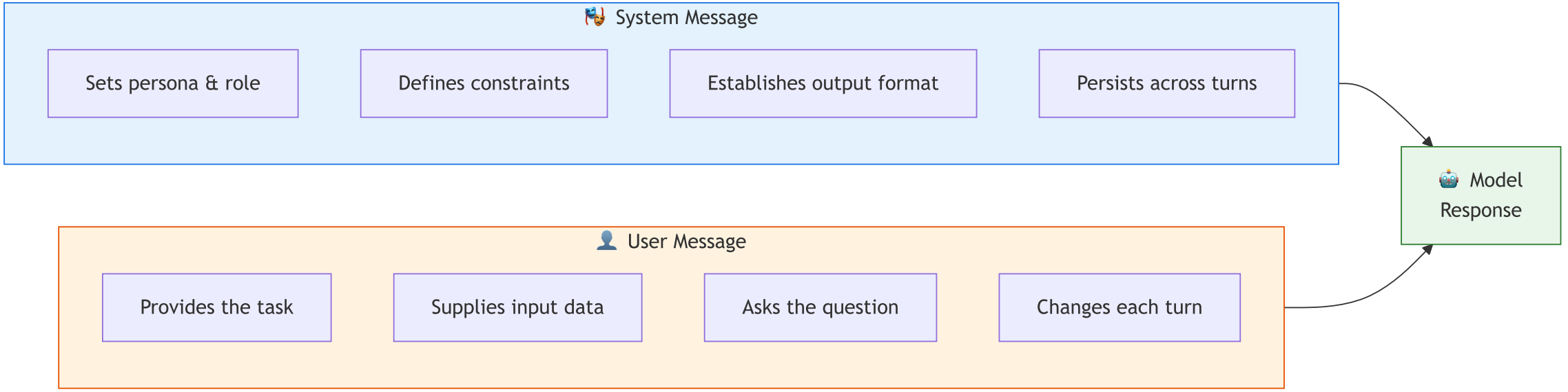
✅
```
"Analyze this data and respond as JSON:
    {
        'trend': 'up|down|stable',
        'key_metric': number,
        'recommendation': 'one sentence'
    }"
```

The System Message vs User Message

Who Does What?

KavinSchool

System vs User: Different Jobs



System Message Best Practices

```
# A well-crafted system message
system = """You are a Python code review assistant for a team
that follows these standards:

ROLE: Senior code reviewer
STYLE: Direct, constructive, specific
FORMAT: Respond as a numbered list of findings
CONSTRAINTS:
- Focus on: bugs, security, performance, readability
- Severity levels: CRITICAL, HIGH, MEDIUM, LOW
- Always suggest a fix, not just the problem
- If the code is good, say so briefly

DO NOT:
- Rewrite the entire function unless asked
- Comment on variable naming unless misleading
- Suggest changes that break the public API"""
```

When to Use System vs User

Content	Put In	Why
Role / persona	**System**	Persists across turns
Output format	**System**	Consistent formatting
Constraints / rules	**System**	Always applied
The actual task	**User**	Changes each request
Input data	**User**	Different every time
Specific questions	**User**	Varies per turn

Rule of thumb: If it should apply to EVERY request, put it in the system message.

Demo Time

Rewriting a Bad Prompt: 5 Iterations Live

KavinSchool

Demo: Prompt Rewriting (5 Iterations)

We'll take this bad prompt and improve it 5 times:

```
"Tell me about Python errors"
```

Iteration 1: Add specificity

Iteration 2: Add context

Iteration 3: Add output format

Iteration 4: Add constraints

Iteration 5: Add examples (few-shot)

> Open `day2/demos/demo-prompt-rewriting.py`

Demo: Watching the Output Improve

Iteration	Change	Quality Leap
v0 (original)	Vague	Generic essay about Python errors
v1 (+specificity)	"Explain the 5 most common..."	Focused, but still wordy
v2 (+context)	"...for a junior developer"	Better tone, simpler language
v3 (+format)	"Respond as a table"	Clean, scannable output
v4 (+constraints)	"One sentence each, with fix"	Concise, actionable
v5 (+example)	Added one example row	Consistent structure

> Each iteration takes seconds. The cumulative improvement is dramatic.

Session 1: Key Takeaways

1. **Good prompts have 4 building blocks.** Instruction, Context, Input Data, Output Format
2. **Start with the goal.** Write expected output first, then craft the prompt
3. **Avoid anti-patterns.** Vague, leading, conflicting, unformatted prompts fail
4. **System message = persistent rules.** User message = per-request task
5. **Iterate your prompts.** Small changes produce big quality improvements

> Next: Session 2, Prompting Techniques. Zero-shot, few-shot, Chain-of-Thought, and more!

Questions?

Session 1 Complete. Break Time!

KavinSchool